

UNIVERSIDAD TECNOLÓGICA NACIONAL
Facultad Regional Mendoza
Tecnicatura Universitaria en Programación - A Distancia

Programación 2

Trabajo Practico Integrador

Food Store - Sistema de Gestión de Pedidos de Comida (Consola + JDBC)

Integrantes:

Sabrina Giménez

Joaquín Godoy

Ares Ocaña

Ismael Saleme

Fecha de entrega: 22 de junio de 2026

Marco Teórico

Java y la Programación Orientada a Objetos

El desarrollo del sistema se realizó en Java 25, utilizando los principios fundamentales de la Programación Orientada a Objetos (POO): encapsulamiento, herencia, polimorfismo y abstracción.

Se definió una clase abstracta Base que centraliza los atributos comunes a todas las entidades del dominio (id, eliminado, createdAt), de la cual heredan Categoria, Producto, Usuario, Pedido y DetallePedido. Esto evita la duplicación de código y refleja el principio de herencia aplicado a un caso real: todas las entidades comparten un ciclo de vida similar (creación, baja lógica, marca temporal).

La interfaz Calculable, implementada por la clase Pedido, define el contrato calcularTotal(), que permite calcular el total de un pedido en base a la sumatoria de los subtotales de sus detalles. Este es un ejemplo de polimorfismo por interfaz: cualquier entidad que necesite exponer un comportamiento de cálculo de totales podría implementar la misma interfaz sin modificar el resto del sistema.

También se utilizaron interfaces genéricas, como IBaseDAO<T>, que define las operaciones CRUD básicas (save, findById, findAll, update, delete) de forma independiente del tipo de entidad concreta. Esto permite que cada DAO específico (CategoriaDAO, ProductoDAO, etc.) implemente el contrato sin repetir la firma de los métodos, aplicando el principio de generics de Java para reducir código repetido.

Bases de datos relacionales y MySQL

La persistencia del sistema se realiza sobre una base de datos relacional MySQL, ejecutada localmente mediante XAMPP. El modelo de datos refleja directamente el diagrama de clases UML provisto por la cátedra: cada entidad del dominio (Categoria, Producto, Usuario, Pedido, DetallePedido) corresponde a una tabla, y las relaciones entre clases (1:N y N:1) se representan mediante claves foráneas.

Se aplicó el concepto de soft delete: en lugar de eliminar físicamente los registros de la base de datos con una sentencia DELETE, se utiliza una columna booleana eliminada que marca el registro como inactivo. Esta decisión permite conservar el historial de datos (por ejemplo, los pedidos de un usuario eliminado siguen existiendo) sin comprometer la integridad referencial del modelo.

JDBC (Java Database Connectivity)

La conexión entre la aplicación Java y la base de datos MySQL se realiza mediante JDBC, utilizando el driver mysql-connector-j. Toda la configuración de conexión se centraliza en una única clase, ConexionDB, que expone un método estático getConnection() y evita que las credenciales de acceso se repitan en distintas partes del código.

Para la ejecución de consultas se utilizó exclusivamente PreparedStatement en lugar de Statement, lo cual previene ataques de inyección SQL al parametrizar los valores en lugar de concatenarlos directamente en la cadena de consulta. El mapeo de resultados se realiza mediante ResultSet, leyendo cada columna y construyendo el objeto de dominio correspondiente.

Todas las conexiones y statements se gestionan mediante el bloque try-with-resources de Java, que garantiza el cierre automático de los recursos (Connection, PreparedStatement) incluso si ocurre una excepción durante su uso, evitando fugas de conexiones abiertas.

Transacciones

La creación de un pedido es la operación más sensible del sistema desde el punto de vista de la integridad de datos, ya que involucra la inserción de un registro en la tabla pedido y, a continuación, de uno o más registros en la tabla detalle_pedido. Para garantizar que esta operación sea atómica, se desactivo el autocommit de la conexión (setAutoCommit(false)) antes de ejecutar ambas inserciones, y se realizó un commit() único al finalizar con éxito. Si ocurre un error durante la inserción de los detalles, se ejecuta un rollback() que revierte también la inserción del pedido, evitando que queden pedidos huérfanos sin detalles en la base de datos.

Patrón DAO (Data Access Object)

Se aplicó el patrón de diseño DAO para separar la lógica de acceso a datos del resto de la aplicación. Cada entidad del dominio cuenta con su propia clase DAO (CategoriaDAO,

ProductoDAO, UsuarioDAO, PedidoDAO), que es la única responsable de construir y ejecutar las sentencias SQL correspondientes. Ninguna otra capa del sistema (ni los servicios ni los menús) contiene código SQL, lo cual facilita el mantenimiento y permite, en teoría, reemplazar el motor de base de datos sin afectar la lógica de negocio.

Arquitectura en capas

El proyecto se organizó en capas claramente diferenciadas, siguiendo la estructura de paquetes recomendada por la cátedra:

- entities: el modelo de dominio, sin lógica de negocio ni acceso a datos.
- dao: único lugar donde se escribe SQL y se utiliza JDBC.
- service: coordinación de casos de uso, validaciones y reglas de negocio.
- menu / Main: interacción con el usuario por consola (Scanner) y construcción de objetos.
- enums: los tipos enumerados del dominio (Rol, Estado, FormaPago).
- exception: excepciones propias del dominio, como EntityNotFoundException.

Esta separación responde al principio de responsabilidad única: cada clase tiene un único motivo para cambiar. Por ejemplo, si se necesitara modificar una validación de negocio, el cambio se realiza únicamente en la capa de servicios, sin tocar el DAO ni el menú.

Decisiones Técnicas y Arquitectura

Organización del proyecto

El proyecto se estructuró como un proyecto Java simple (sin gestor de dependencias como Maven o Gradle), agregando el driver JDBC de MySQL manualmente como librería del proyecto en IntelliJ IDEA. Esta decisión se tomó por simplicidad, dado que el equipo de trabajo no contaba con experiencia previa en herramientas de gestión de dependencias, y el proyecto solo requiere una única librería externa.

La estructura final de paquetes quedó organizada de la siguiente manera:

```
src/
├── Main.java
├── config/ (ConexionDB)
```

```

|— entities/  (Base, Categoria, Producto, Usuario,
|              Pedido, DetallePedido, Calculable)
|— enums/    (Rol, Estado, FormaPago)
|— dao/      (IBaseDAO + un DAO por entidad)
|— service/  (logica de negocio y validaciones)
|— menu/     (interaccion por consola)
|— exception/ (EntityNotFoundException)

```

División del trabajo en equipo

Dado que el trabajo se realizó en un equipo de cuatro personas con disponibilidad de tiempo limitada y desigual, se decidió dividir el desarrollo en franjas verticales completas por entidad (entidad, DAO, servicio y menú), en lugar de dividir por capas horizontales. Esta decisión se tomó específicamente para minimizar la coordinación necesaria entre los integrantes: cada persona pudo trabajar en sus propios archivos sin depender de que otro integrante terminara su parte en simultáneo.

Para evitar que el equipo quedara bloqueado esperando una base de código compartida, un integrante desarrollo primero, de forma individual, toda la infraestructura común: la clase abstracta Base, los enums, la interfaz Calculable, la interfaz genérica IBaseDAO, la clase ConexionDB y el script de creación de la base de datos. Junto con esto, se desarrolló también una implementación completa y funcional de la entidad Categoría, que sirvió como plantilla de referencia para que el resto del equipo replicara el mismo patrón de código en sus respectivas entidades.

Como consecuencia de este orden de trabajo, el módulo de Pedido —que depende de Usuario y Producto para poder probarse— se desarrolló inicialmente contra datos de prueba hardcodedos en el propio menú (un usuario y dos productos ficticios), de forma de no bloquear el avance mientras el módulo de Usuario aún estaba en desarrollo. Una vez integrados los módulos de Usuario y Producto, estos datos de prueba fueron reemplazados por las consultas reales a la base de datos, permitiendo seleccionar usuarios y productos existentes desde el menú de creación de pedidos.

Manejo de relaciones entre entidades

Para las relaciones entre entidades (por ejemplo, Producto con Categoría, o Pedido con Usuario), se optó por no realizar consultas JOIN en las sentencias SQL. En su lugar, los métodos de mapeo de cada DAO (mapRow) construyen el objeto relacionado utilizando únicamente el identificador (id) obtenido de la clave foránea, y -en los casos donde se necesitaba mostrar información completa del objeto relacionado, como el nombre de la categoría de un producto- se realizó una consulta adicional al DAO correspondiente desde dentro del método de mapeo.

Esta decisión simplificó la escritura de las consultas SQL a costa de una pequeña pérdida de eficiencia (una consulta adicional por cada fila mapeada). Para el alcance de este trabajo práctico, se consideró una solución aceptable, priorizando la claridad del código sobre la optimización de consultas.

Validaciones de negocio implementadas

Además de las operaciones CRUD básicas, se implementaron las siguientes reglas de negocio en la capa de servicios:

- No se permite crear un producto con precio o stock negativo.
- No se permite crear un producto asociado a una categoría inexistente o eliminada.
- El mail de un usuario debe ser único en el sistema; se valida tanto en la creación como en la edición.
- No se permite crear un pedido sin un usuario asociado.
- No se permite agregar un detalle de pedido con una cantidad menor o igual a cero.
- No se permite agregar a un pedido una cantidad de un producto superior al stock disponible de ese producto.
- Todas las bajas (categorías, productos, usuarios, pedidos) son lógicas, mediante el campo eliminado, nunca se ejecuta un DELETE físico.

Capturas de pantalla del sistema funcionando

A continuación se incluyen capturas de pantalla que muestran el funcionamiento del sistema desde la consola, recorriendo los flujos principales de cada módulo.

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===  
1. Categorías  
2. Productos  
3. Usuarios  
4. Pedidos  
0. Salir  
Seleccione: |
```

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
```

```
1. Categorías
```

```
2. Productos
```

```
3. Usuarios
```

```
4. Pedidos
```

```
0. Salir
```

```
Seleccione: 1
```

```
=== CATEGORÍAS ===
```

```
1. Listar
```

```
2. Crear
```

```
3. Editar
```

```
4. Eliminar
```

```
0. Volver
```

```
Seleccione: 2
```

```
Nombre: Cate3
```

```
Descripción: Una descripción
```

```
Categoría creada con ID: 3
```

```
=== CATEGORÍAS ===
```

```
1. Listar
```

```
2. Crear
```

```
3. Editar
```

```
4. Eliminar
```

```
0. Volver
```

```
Seleccione: 1
```

```
[1] Cate - Cate
```

```
[2] Cate2 - aa
```

```
[3] Cate3 - Una descripción
```

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 2

=== PRODUCTOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: prod3
Descripcion: Una descripción
Precio: 150
Stock: 5
Imagen (URL o nombre, Enter para omitir):
Disponible (S/N): s
ID de categoría: 3
Producto creado con ID: 3

=== PRODUCTOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
[1] prod1 - $100.00 | Stock: 1 | Categoría: Cate | Disponible: Si
[2] prod2 - $50.00 | Stock: 1 | Categoría: Cate | Disponible: Si
[3] prod3 - $150.00 | Stock: 5 | Categoría: Cate3 | Disponible: Si
```

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===  
1. Categorías  
2. Productos  
3. Usuarios  
4. Pedidos  
0. Salir  
Seleccione: 3  
  
=== USUARIOS ===  
1. Listar  
2. Crear  
3. Editar  
4. Eliminar  
0. Volver  
Seleccione: 1  
[1] Ares Ocana - aaa@gmail.com | Rol: ADMIN
```

```
=== PEDIDOS ===
1. Listar
2. Crear Pedido
3. Editar Estado
4. Eliminar
0. Volver
Seleccione: 2

--- Usuarios disponibles ---
[1] Ares Ocana - aaa@gmail.com | Rol: ADMIN
ID de usuario: 1
Forma de pago (TARJETA/TRANSFERENCIA/EFFECTIVO): tarjeta

--- Productos disponibles ---
[1] prod1 - $100.00 | Stock: 1 | Categoria: Cate | Disponible: Si
[2] prod2 - $50.00 | Stock: 1 | Categoria: Cate | Disponible: Si
[3] prod3 - $150.00 | Stock: 5 | Categoria: Cate3 | Disponible: Si
ID de producto: 1
Cantidad: 1
Agregar otro producto (S/N): s

--- Productos disponibles ---
[1] prod1 - $100.00 | Stock: 1 | Categoria: Cate | Disponible: Si
[2] prod2 - $50.00 | Stock: 1 | Categoria: Cate | Disponible: Si
[3] prod3 - $150.00 | Stock: 5 | Categoria: Cate3 | Disponible: Si
ID de producto: 3
Cantidad: 2
Agregar otro producto (S/N): n
Pedido registrado con exito.
Total calculado: $400.0
```

```
=== PEDIDOS ===
1. Listar
2. Crear Pedido
3. Editar Estado
4. Eliminar
0. Volver
Seleccione: 1
ID: 1 | Fecha: 2026-06-22 | Estado: CONFIRMADO | Total: $0.0
ID: 2 | Fecha: 2026-06-22 | Estado: PENDIENTE | Total: $100.0
ID: 3 | Fecha: 2026-06-22 | Estado: PENDIENTE | Total: $400.0
```

Dificultades y Soluciones

Coordinación del trabajo en equipo bajo restricciones de tiempo

La principal dificultad del trabajo práctico no fue técnica, sino organizativa: el equipo contó con poco tiempo y disponibilidad desigual entre sus integrantes para coordinar el desarrollo de forma sincrónica. Se resuelve adoptando una división del trabajo que minimizará la necesidad de coordinación continua, descrita en la sección de Decisiones Técnicas, concentrando el desarrollo de la infraestructura compartida en una sola persona y al comienzo del proyecto, de forma de no generar bloqueos cruzados entre los integrantes durante el resto del desarrollo.

Dependencias entre módulos desarrollados en paralelo

El módulo de Pedido depende, tanto en su modelo de datos como en su flujo de uso, de la existencia de usuarios y productos reales. Como estos módulos se desarrollaron en paralelo por otros integrantes del equipo, no era posible probar el flujo completo de creación de un pedido hasta que ambos estuvieran terminados. Se resolvió utilizando objetos de prueba hardcodeados (un usuario y dos productos ficticios) dentro del propio menú de Pedido, lo cual permitió desarrollar y probar de forma aislada la lógica de cálculo de totales y la transacción de persistencia. Una vez integrados los módulos de Usuario y Producto al repositorio, se reemplazaron los datos de prueba por consultas reales a través de los servicios correspondientes.

Inconsistencia entre el orden de parámetros de constructores y servicios

Durante la integración del módulo de Producto, se detectó un error de compilación provocado por una inconsistencia entre el orden de los parámetros definidos en el constructor de la clase Producto y el orden en que estos eran pasados desde ProductoService al instanciar el objeto. El error fue identificado por el mensaje de la propia herramienta de desarrollo (IntelliJ IDEA), que señalaba una discrepancia de tipos entre los parámetros precio y descripcion. Se resolvió modificando el orden de los argumentos en la llamada al constructor desde el servicio, dejando como criterio que la firma del constructor de la entidad es la fuente de verdad sobre la cual deben adaptarse las demás capas.

Categoría nula al listar productos

Al implementar la asociación entre Producto y Categoría sin utilizar una consulta JOIN, se detectó que, si bien el producto se guardaba correctamente con su categoria_id en la base de datos, al listar los productos la categoría asociada se mostraba como nula en la consola. La causa del problema fue que el método de mapeo del DAO (mapRow) únicamente asignaba el identificador de la categoría al objeto, sin cargar sus demás atributos (como el nombre), por lo que el método toString() del producto no podía mostrar información legible de la categoría. Se resolvió realizando, dentro del propio método de mapeo de ProductoDAO, una consulta adicional a CategoríaDAO para obtener el objeto Categoría completo a partir de su identificador.

Falta de validación de stock al crear un pedido

Se detectó que el sistema permitía crear un detalle de pedido con una cantidad de producto superior al stock disponible, lo cual no tiene sentido desde el punto de vista del dominio del negocio, aún cuando la consigna no lo mencionaba de forma explícita. Se resolvió agregando una validación en el menú de creación de pedidos, en el momento en que se solicita la cantidad de cada producto, comparándola contra el stock disponible del producto seleccionado y rechazando la operación si la cantidad solicitada lo supera.

Decisión de no descontar stock automáticamente

Se evaluó la posibilidad de descontar automáticamente el stock de un producto al confirmar un pedido. Se decidió no implementar este comportamiento, dado que no estaba contemplado en los criterios de aceptación de las historias de usuario provistas por la cátedra, y su implementación

hubiera requerido definir reglas adicionales no especificadas (por ejemplo, si el stock debía descontarse al crear el pedido o al confirmarlo, y si debía restituirse en caso de cancelación). Se prefirió mantener el stock como un campo gestionado manualmente desde el módulo de Productos, evitando introducir lógica de negocio no solicitada a esta etapa del desarrollo.

Bibliografía / Webgrafía

- Documentación oficial de Java SE - Oracle. <https://docs.oracle.com/en/java/javase/>
- Documentación oficial de JDBC - Oracle. <https://docs.oracle.com/javase/tutorial/jdbc/>
- Documentación oficial de MySQL Connector/J. <https://dev.mysql.com/doc/connector-j/en/>
- Documentación oficial de MySQL. <https://dev.mysql.com/doc/>
- Documentación de XAMPP. <https://www.apachefriends.org/>
- Material de cátedra: Consigna del Trabajo Práctico Integrador - Programación 2, UTN Mendoza.

Se utilizaron como apoyo durante el desarrollo las herramientas de inteligencia artificial Claude (Anthropic), ChatGPT (OpenAI) y Gemini (Google), principalmente para resolver dudas de lógica y sintaxis, depurar errores de compilación y de ejecución, y revisar la consistencia del código entre los distintos módulos desarrollados por el equipo.

Enlaces

Repositorio del proyecto: <https://github.com/aresito22/TPI-Programacion2-2026>